baseten

ИССЛЕДОВАНИЕ 29 ОКТЯБРЯ 2025 ГОДА

Уделяйте больше внимания стратегии, а не токенам: ускорьте обучение с помощью явных рассуждений в рамках RGT (Rationale-Guided Training)

Обучайте тому, почему, а не только тому, что: Rationale-Guided Training

 Чарльз О'Нил (Базеттен)

 Гарри Партридж (Бастетен)

 Мудит Джаясекара (Басетен)

TL;DR

Мы представляем Rationale-Guided Training (RGT) — метод обучения, который повышает эффективность использования обучающих данных по сравнению с традиционными подходами за счет того, что во время обучения скрытые стратегии логического вывода становятся явными. В отличие от стандартной тонкой настройки с учителем или обучения с подкреплением, которые учат модели только тому, *что* нужно выдавать, RGT учит модели тому, *почему* те или иные результаты являются правильными, преобразуя обоснования оценки в явные стратегические маркеры, которые управляют генерацией.

Обучение моделей искусственного интеллекта требует больших затрат — не только на вычислительные ресурсы, но и на данные. Каждый пример требует затрат на создание, разметку и проверку, особенно для задач, которые невозможно проверить. Что, если бы мы могли добиться такой же производительности модели, используя в 3–5 раз меньше обучающих примеров? Мы разработали удивительно простой метод, который позволяет добиться именно этого. Обучая модели тому, *почему* ответы являются правильными (в частности, делая явными скрытые стратегии, которые

помогают решать задачи в процессе обучения), мы достигаем тех же показателей производительности, используя в разы меньше данных. Мы добились этого без использования новых архитектур или сложных систем обучения с подкреплением. Просто мы по-новому взглянули на структуру обучающих данных.

Проблема стратегии: почему одного бита недостаточно

Рассмотрим, как студент учится вычислять этот интеграл: $\int x \cos(x)\, dx$

Текущий подход к обучению с подкреплением: учащийся пробует разные подходы. Может быть, он попытается подставить вместо интеграла сумму, может быть, разложит на множители, может быть, воспользуется тригонометрическим тождеством. В конце концов он натыкается на метод интегрирования по частям. Он получает сигнал о вознаграждении: «правильно» или «неправильно». Один бит информации на весь путь решения.

Современный подход к обучению с помощью SFT: мы показываем им тысячи правильно решенных интегралов. Увидев достаточное количество примеров, в которых $\int x \cos(x)\, dx = x \sin(x) + \cos(x) + C$, они запоминают закономерность. Но научились ли они *когда* применять метод интегрирования по частям, а когда — подстановку?

Как на самом деле учатся люди: учащийся вырабатывает скрытую стратегию («Я попробую интегрировать по частям, потому что у меня есть полином, умноженный на тригонометрическую функцию»), следует этой стратегии и получает обратную связь о том, был ли этот стратегический выбор правильным. Суть в том, что стратегия — это скрытая переменная, которая определяет весь путь решения. Как только вы понимаете, что здесь нужно использовать метод интегрирования по частям, все остальное становится механическим процессом.

В этом и заключается фундаментальная неэффективность современного обучения: модели должны реконструировать эти скрытые стратегии только на основе примеров. Если модель видит 500 примеров, в которых метод интегрирования по частям работает для полиномиальных и тригонометрических комбинаций, она может вывести это стратегическое правило. А может и не вывести, даже если увидит 5000 примеров. Или же она может обнаружить ложные корреляции.

Математическое объяснение того, почему это важно, довольно простое. При стандартном обучении мы просим модели изучить $P(y \mid x)$, где y — это полное решение. Но существует скрытая стратегическая переменная z, такая что $P(y \mid x) = \sum_z P(y \mid x, z)\, P(z \mid x)$. Когда z является правильной стратегией, $P(y \mid x, z)$ часто бывает

практически детерминированной. Самое сложное — выбрать z, а не реализовать ее. Делая z явной, мы напрямую обучаем $P(z \mid x)$, а не надеемся, что модель выведет ее из предельного распределения.

Проблема с точкой шарнира

Вот что усугубляет ситуацию: в любом решении есть *ключевые моменты* — конкретные этапы, на которых проявляется стратегическое решение. В своей недавней работе компания Thinking Machines называет их «точками принятия решений». Остальная последовательность действий может быть в значительной степени механической, но именно эти критические моменты определяют успех или провал.

Рассмотрим модель, которая расшифровывает медицинские записи и неправильно добавляет «диабетическую гипертензию», когда в записи упоминается только «гипертензия». Ключевой момент — это токен, в котором генерируется слово «диабетическая». В выводе из 500 токенов важно только это решение. Однако современные методы обучения распределяют баллы поровну между всеми 500 токенами.

Итеративная SFT (наш текущий передовой метод) частично решает эту проблему: генерируем результат, просим специалиста по оценке выявить ошибки, дорабатываем результат до идеального состояния, а затем обучаем модель на идеальном результате. (Конечно, нужны хорошие специалисты по оценке, особенно для задач, которые невозможно проверить. Мы много работали над этим вопросом; см. здесь наше мнение на этот счет). Это работает: именно так мы уже некоторое время обучаем модели в Baseten. Но при этом теряется самая ценная информация: *где* произошла ошибка и *почему* она возникла.

Think about the information we're discarding:

- The grader knows exactly which tokens were problematic (the hinge points)
- The grader has a rationale ("Don't infer chronic conditions from medications alone")
- This same principle applies across hundreds of different surface forms

Instead of training the model to implicitly discover that “diabetic” was the problem token by seeing enough examples with and without it, why not just tell the model directly? This is the difference between hoping a pattern emerges from statistical regularities versus explicitly teaching the causal structure of the task.

The Evolution from SFT to iSFT: Getting Closer, But Not There Yet

The simplest approach to training is supervised fine-tuning (SFT) on outputs from a larger model. Take GPT-5's outputs, train your smaller model to mimic them. It works, but with an obvious ceiling: your model can never exceed the teacher. You're learning to copy surface patterns, not understand underlying principles. Worse, you inherit all the teacher's errors and biases without any mechanism to improve upon them.

Iterative SFT (iSFT) breaks through this ceiling. Instead of training on raw teacher outputs, we refine them to perfection first:

1. Generate initial output y_0 from the model
2. Have a grader evaluate and identify errors (a grader outputs a rationale as well as a score, usually binary)
3. Refine iteratively to $y_1, y_2, \ldots$ until reaching a perfect output y^*
4. Train on $(x \to y^*)$

This is powerful for three reasons. First, we get dense supervised signal without the complexity of RL, as every example yields the "right" gradients. Second, by training only on grader-perfect outputs, we push the model toward outputs that satisfy our actual task constraints, not just outputs that look like the teacher. Third, it delivers monotonic improvement with dataset size and works cleanly with standard training infrastructure.

From an information-theoretic view, iSFT is learning the forward KL divergence to the distribution of acceptable outputs. Given the set $\mathcal{G}(x)$ of all grader-acceptable outputs for input x, we're minimizing:

$$\mathbb{E}_x[\mathrm{KL}(p^*(\cdot|x)|p_\theta(\cdot|x))]$$

where $p^*(y|x) \propto 1_{y \in \mathcal{G}(x)}$. In the limit of infinite data and compute, this converges to the same distribution as policy optimization with a binary reward. It's RL without the RL machinery.

But iSFT still only uses the *final* corrected sequence and discards the richest parts of the refinement process:

- **Where the fix happened:** The diff between y_0 and y^* pinpoints exactly which tokens mattered. iSFT spreads credit over the entire sequence, diluting gradient on the

decisive edits.

- **Why the fix was needed:** The grader's rationale contains the latent rule that caused the correction ("don't infer diagnoses from medications"). iSFT never conditions on this causal variable.

- **How to generalize the fix:** The same latent rule applies across thousands of surface forms. Training only on $x \to y^*$ forces the model to re-learn that rule piecemeal in every context.

Think about it: we spend compute generating grader feedback that identifies exactly what went wrong and why. We use this to create perfect outputs. Then we throw away everything except the final output and train on that. It's like having a tutor who identifies your specific misconceptions, helps you fix them, but then only lets you study the corrected work without the explanations.

This sets up the key insight: what if we kept those explanations?

Rationale-Guided Training: Making the Implicit Explicit

The solution is almost embarrassingly simple. Instead of hoping models infer strategies from examples, we just tell them the strategies directly.

Here's RGT in its entirety. For each training example, instead of creating one input-output pair, we create two:

- **Pair A (standard):** `prompt → refined_output`

- **Pair B (strategy-explicit):** `prompt → [THINK] strategy [/THINK] refined_output`

That's it. No new architecture. No reinforcement learning machinery. Just structured supervised learning with strategy tokens that make the latent reasoning explicit.

Let's make this concrete. In our medical transcription example:

Traditional training:

```
Input: "Patient has hypertension, on metformin..."
Output: "Diagnoses: Hypertension"
```

RGT training (both pairs):

```
Pair A: Input → "Diagnoses: Hypertension"
Pair B: Input → "[THINK] metformin indicates diabetes management but
diagnosis not stated explicitly; only include confirmed conditions [/THINK]
Diagnoses: Hypertension"
```

The model trains on both simultaneously. During inference, we prepend empty think tags and the model performs well even without explicit reasoning. It has internalized the strategies.

What goes in those strategy tokens? We distill them from the grader feedback we're already generating (we typically use a small, lightweight model like Claude Haiku with an optimized meta-prompt to do this). When our grader says "Added 'diabetic hypertension' not mentioned in transcript," we transform this into a compact strategic rule:

```
mistake: "Inferred diagnosis from medication"
guardrail: "Only include explicitly stated diagnoses"
```

This addresses every inefficiency we identified. The strategy tokens explicitly mark where and why the critical decision occurs. Instead of diluting gradient across 500 tokens, the model learns that the strategic decision about "diabetic" is what matters. We're now no longer asking the model to marginalize over all possible strategies to learn $P(y|x)$. We're directly teaching $P(y|x, z)$ where z is explicit. Since executing a known strategy is often nearly deterministic, particularly when the model knows how to roughly do the task already (ie write a clinical note, solve integration by parts, etc.), this dramatically reduces the learning complexity. Finally, a single strategy ("don't infer diagnoses from medications") applies to thousands of surface variations. By making it explicit once, we don't need to re-learn it for every variation of metformin, insulin, statins, or any other medication.

The beauty is that we're not changing the training infrastructure at all. It's still supervised learning with cross-entropy loss. We're just being smarter about what we supervise on. The compute cost is virtually identical. For iSFT, we're already evaluating model outputs in order to refine them. So we just use a small model to distill these evaluations down into a strategic rule for that output - making the latent variable explicit.

You can think of this as the training-time analog of chain-of-thought prompting. But instead of hoping the model generates good reasoning at inference time, we're teaching it *which* reasoning strategies lead to correct outputs during training. The model learns not just what to output, but why that output is correct given the strategic context.

Results

We evaluated RGT on two real-world medical scribing tasks in active production use: dental clinical notes (Task A) and emergency department notes (Task B). Both tasks use custom graders developed over several weeks with domain experts, scoring outputs on 6-point and 5-point scales respectively. All experiments use Gemini-2.5-Pro for data generation, evaluation and refinement, Claude 3.5 Haiku for distilling strategic rules, and Qwen3-32B as the model we're training. We train with LoRA adapters, rank 256, with alpha=32, a constant learning rate (lr=2e-4), and zero LoRA dropout.

At 25,000 training examples, each method plateaus at distinct performance levels: standard SFT achieves 52-56% normalised accuracy, iterative SFT reaches 66-78%, and RGT climbs to 72-83% across both tasks. The 20-27 percentage point gap between RGT and standard SFT represents a difference in learning efficiency; RGT has extracted far more knowledge from the same amount of data. The optional thinking mode provides a modest additional boost (1-2 points), suggesting the primary value of strategic training lies in what the model internalises, not what it explicitly reasons through.

Figure 1: Final model performance after 25,000 training examples across dental (Task A) and emergency (Task B) clinical scribing tasks. RGT achieves 72-83% accuracy compared to 52-56% for standard SFT.

The learning curves reveal that RGT's advantage emerges immediately. Within the first 1,000 examples, RGT achieves performance levels that take iSFT 5,000 examples and standard SFT over 10,000 examples to reach. Once established, these performance gaps remain remarkably stable throughout training, with all three methods showing similar learning rates after initial plateauing.

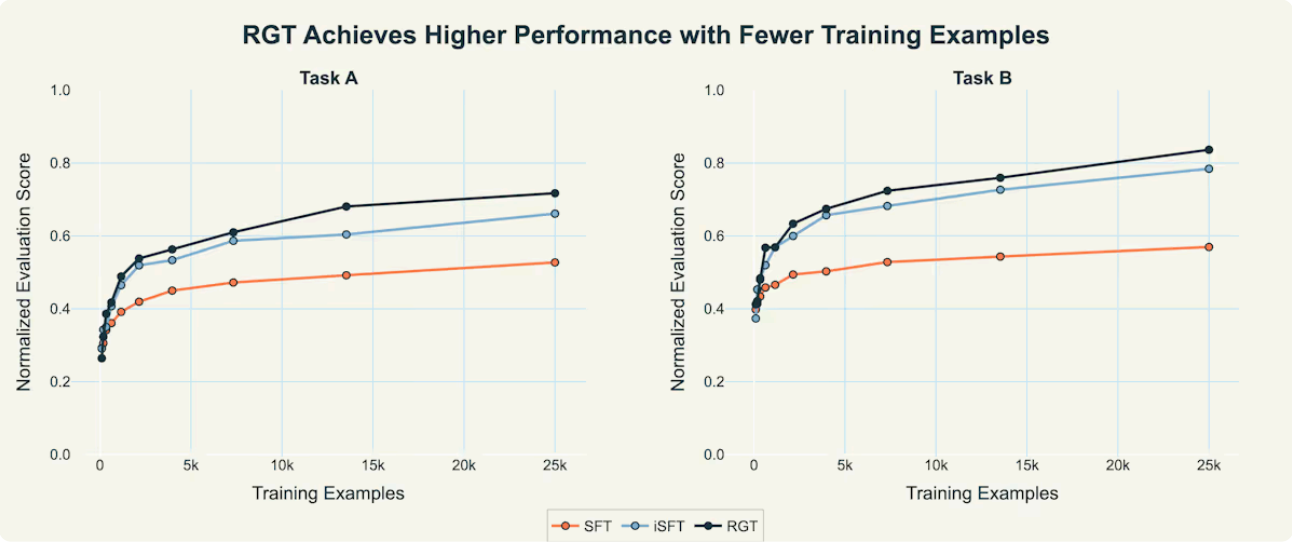


Figure 2: Learning curves showing normalized evaluation scores across training checkpoints. RGT demonstrates superior sample efficiency from the start, maintaining a consistent advantage over iSFT and standard SFT throughout training.

While RGT can optionally generate explicit reasoning during inference, the thinking mode provides only marginal gains; the curves track nearly identically throughout training with just a 1-2 percentage point advantage. This probably indicates that RGT's power comes from internalising strategic knowledge during training rather than from explicit chain-of-thought reasoning at inference time, making the method practical for latency-sensitive production deployments. (In order to run without thinking, we simply prepend empty `<think>` tags to the start of generation, which is the format the model has been trained on for the non-thinking examples.)

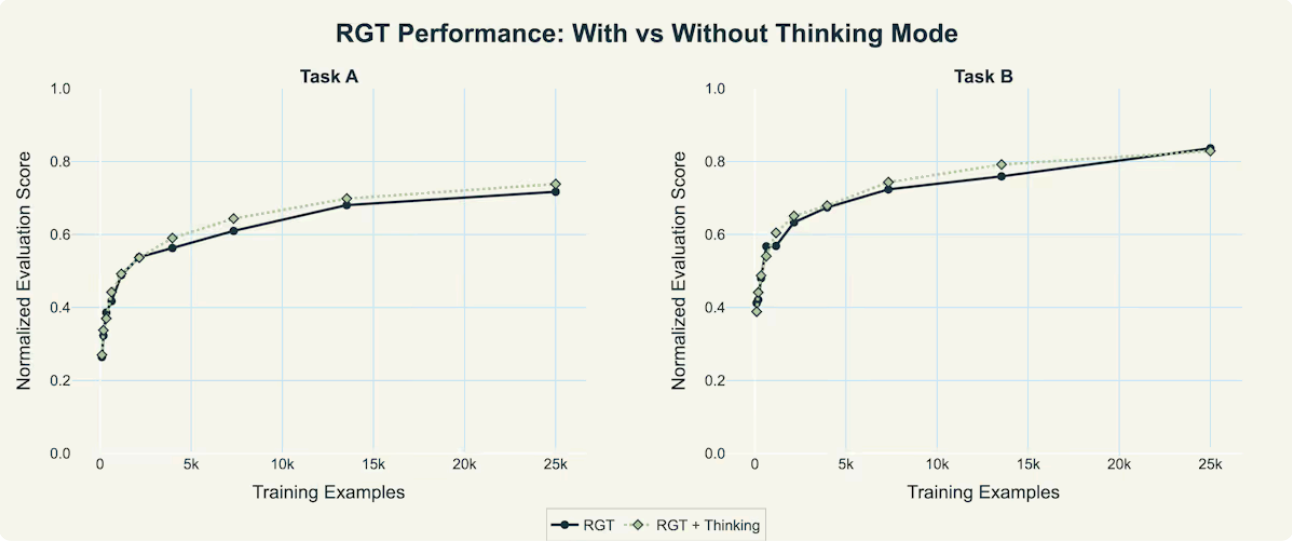


Figure 3: Performance comparison of RGT with and without explicit thinking mode during inference. The minimal gap between modes shows that strategic knowledge is effectively internalised during training.

To quantify sample efficiency, we measured how many training examples each method requires to achieve a 25% improvement in normalised evaluation score—a meaningful performance gain in production settings. We fitted logarithmic growth models to the learning curves and used binary search with linear interpolation to precisely identify where

each method crosses this threshold, normalising all values relative to RGT's requirement. The result is that standard SFT requires over 10x more training data than RGT to reach the same performance milestone, while iSFT falls between at 1.5-1.7x, confirming that explicit strategy training fundamentally changes the data requirements for model improvement.

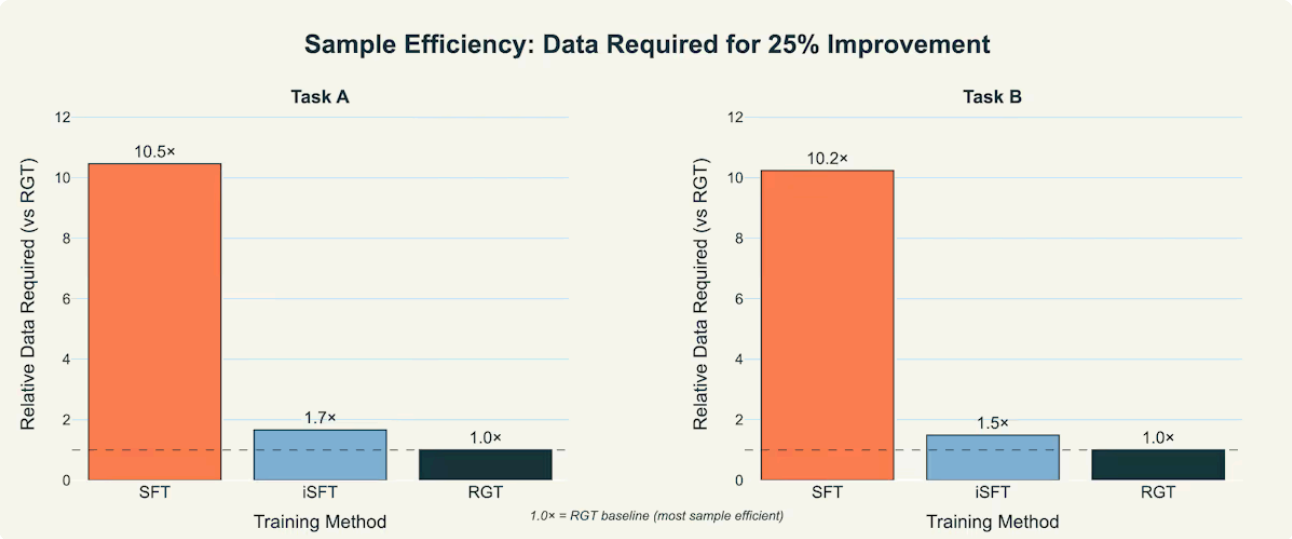


Figure 4: Relative data requirements to achieve 25% performance improvement from baseline. RGT requires 10x fewer examples than standard SFT to reach the same performance target.

These aren't synthetic benchmarks but rather are production systems where every point of accuracy translates to reduced human review time and better patient care. For Task B (emergency notes), jumping from 56% to 83% accuracy means cutting error rates nearly in half. Achieving this with 10x less training data fundamentally changes the economics of deploying these systems.

The consistency across both tasks suggests this isn't task-specific. RGT delivers similar improvements regardless of different grading scales, different medical specialties, different types of clinical documentation. The method appears to capture something fundamental about how to teach models complex tasks efficiently.

Why Does RGT Work?

The core limitation of reinforcement learning with binary rewards is stark: no matter how complex your output, you get at most one bit of information per episode. Consider a 500-token medical note. The model makes 500 separate decisions: which diagnoses to include, how to phrase symptoms, what medications to list. A binary reward provides a single scalar: "good" or "bad." This one bit of feedback must somehow inform all 500 token-level decisions.

Mathematically, the policy gradient update takes the form $\nabla_\theta J = \sum_{t=1}^{T} \nabla_\theta \log p_\theta(y_t|x, y_{<t}) \cdot (r - b)$ where $(r - b)$ is the advantage. The critical observation is that every gradient term receives the exact same scalar weight. The information content of this update is bounded

by the entropy of the binary reward: $h(\alpha) = -\alpha \log_2 \alpha - (1-\alpha)\log_2(1-\alpha)$ where α is the probability of success. This maxes out at exactly one bit when $\alpha = 0.5$ and is strictly less otherwise.

For our 500-token medical note, this means we're spreading one bit of learning signal across 500 decisions. The model has no idea which tokens were problematic—was it the spurious “diabetic” modifier? The missing medication? The incorrect formatting? Every token's gradient gets multiplied by the same "this was wrong" signal, diluting the learning by a factor of 500.

Why Oracle Supervision Still Leaves Information on the Table

Iterative SFT dramatically improves on this by providing the corrected sequence y^*. Now we get $O(T)$ bits of information—supervision for each token position. The gradient becomes $\nabla_\theta J = \sum_{t=1}^{T} \nabla_\theta \log p_\theta(y_t^* | x, y_{<t}^*)$, where each term provides specific guidance about what token should appear at that position.

But here's the subtle inefficiency: the model is learning $P(y|x)$ when what it really needs to learn is $P(y|x,z)$ where z is the latent strategy that determines correctness. The model must marginalize over all possible strategies to learn the output distribution: $P(y|x) = \sum_z P(y|x,z)P(z|x)$. If you have access to the latent variable z, you're always going to be a more efficient learner.

Think about our medical transcription task. The model sees hundreds of examples where medications aren't interpreted as diagnoses. Eventually, it might infer the rule. But it's reverse-engineering this strategic knowledge from statistical regularities rather than learning it directly. This is why iSFT still requires thousands of examples to learn patterns that human medical scribes grasp from a single explanation: “Only include diagnoses explicitly stated by the clinician.”

RGT breaks through this ceiling by making the latent strategy z explicit. Instead of forcing the model to infer strategies from examples, we directly teach $P(y|x,z)$ alongside $P(y|x)$. By conditioning on z, we reduce the conditional entropy from $H(Y|X)$ to $H(Y|X,Z)$. When the strategy is known, generating the correct output often becomes nearly deterministic—the hard part was knowing which rule to apply, not executing it. One strategic rule (”don't infer diagnoses from medications”) prevents the same error across thousands of surface variations: metformin, insulin, statins, ACE inhibitors, and every other medication that might appear.

The strategy tokens also solve the credit assignment problem. Instead of spreading gradient uniformly across 500 tokens, the model learns that the strategic decision at the

hinge point (whether to add "diabetic") is what matters. The gradient concentrates where it's needed most.

Relation to On-Policy Distillation

Thinking Machines' on-policy distillation offers a compelling alternative: instead of binary sequence-level rewards, use a teacher model to provide per-token KL rewards. This gives dense supervision—the student learns to mimic the teacher's distribution at every position. It's elegant and effective.

But RGT goes further. On-policy distillation is fundamentally limited by the teacher's knowledge. You can approach the teacher's performance but never exceed it. RGT, by contrast, extracts strategic rules from grader feedback that may capture patterns the original teacher missed.

This is the difference: we're learning the causal structure of what makes outputs correct rather than just mimicking outputs. A well-calibrated grader that checks specific medical accuracy criteria provides information that no teacher model contains in its logits alone. The grader might enforce constraints like "medication dosages must match standard ranges" or "temporal sequences must be internally consistent", rules that even large teacher models violate. By distilling these rules into strategic tokens, RGT teaches models to be more correct than their teachers.

Moreover, RGT requires no online teacher during training. On-policy distillation needs the teacher model available for every training step to compute KL rewards. RGT distills strategies once and trains offline with standard SFT infrastructure. The computational savings are substantial.

Conclusion

A note on compute costs

Before celebrating these results, we should acknowledge an important caveat: comparing sample efficiency across these methods isn't entirely fair. Both iSFT and RGT require additional compute beyond standard SFT ie running grader evaluations, iteratively refining outputs, and in RGT's case, distilling strategic rules. This represents substantial computational overhead compared to simply training on raw model outputs.

However, the step from iSFT to RGT is essentially free. We're already running graders and generating refinements for iSFT. RGT just adds a lightweight distillation step (using Claude 3.5 Haiku in our experiments) to extract strategic rules from grader feedback we're already producing. The training itself remains standard supervised learning. So while both

advanced methods cost more than basic SFT, RGT delivers 1.5-1.7x better sample efficiency than iSFT at virtually no additional computational cost.

The Deeper Lesson

RGT works because it respects a fundamental truth about learning: not all information is created equal. A binary “right/wrong” signal contains one bit. A corrected sequence contains hundreds of bits. But a strategic rule (”only include explicitly stated diagnoses”) contains the causal structure that makes thousands of outputs correct or incorrect.

Current training methods systematically discard this causal information. They force models to reverse-engineer strategies from statistical patterns, like asking someone to derive physics from watching enough YouTube videos. RGT simply stops throwing this information away.

The implications extend beyond our medical transcription experiments. Any task with clear correctness criteria (code that must compile, legal documents that must satisfy requirements, financial reports that must balance) could benefit from making strategic knowledge explicit during training. We're not just teaching models what to output, but why those outputs are correct.

Looking Forward

We like RGT because it’s practical, yes, but it's also nice that we can finally teach models the way we teach humans: with explanations rather than only examples. When a model makes an error, we can identify the strategic misconception, correct it once, and prevent that entire class of errors going forward.

We are currently using RGT to optimise models for our customers at Parsed. The method requires no new infrastructure, no architectural changes, and no complex RL pipelines. Just a simple insight: if you already know why outputs are correct, teach that knowledge directly.

Sometimes the best ideas are embarrassingly simple. RGT is one of them.

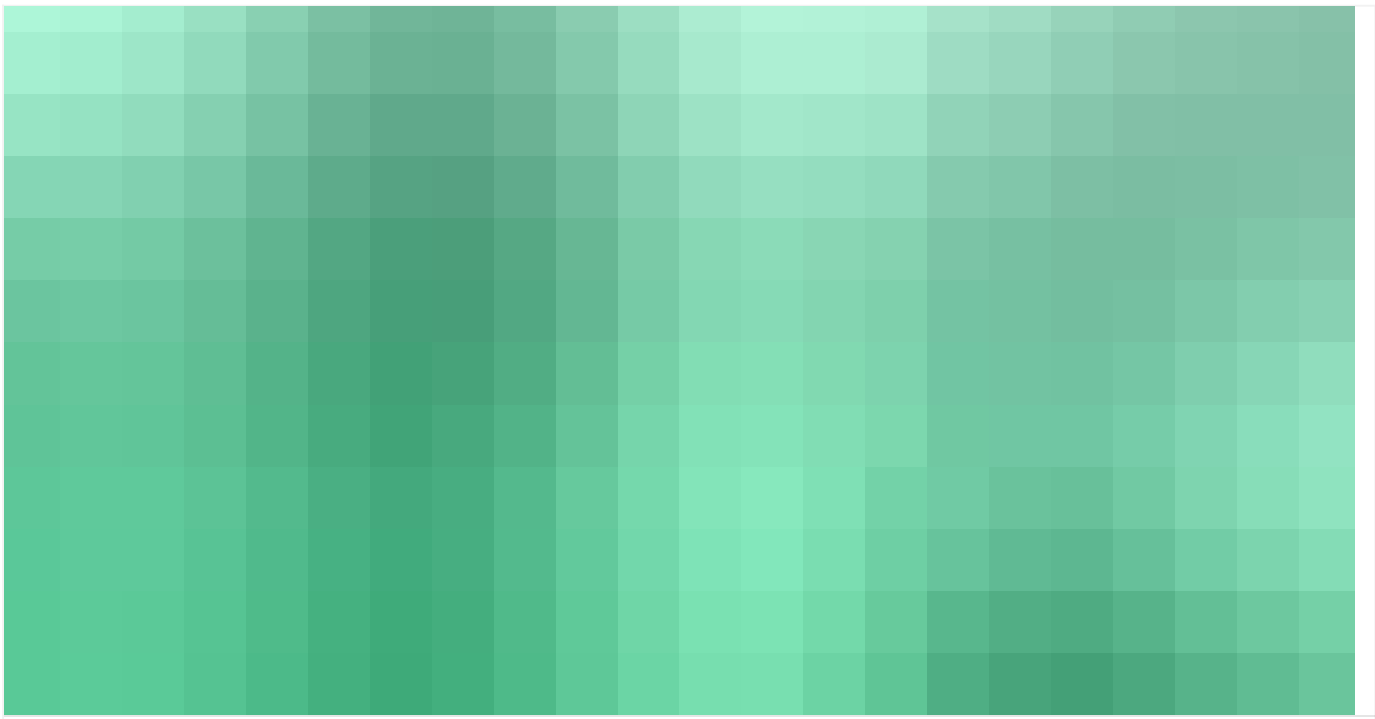
For attribution in academic contexts, please cite this work as:

O'Neill, Charles, Harry Partridge, and Mudith Jayasekara. "Upweight the Strategy, Not the Tokens: Faster Training with Explicit Reasoning Through RGT (Rationale-Guided Training)." Baseten Research, October 29, 2025. https://www.baseten.co/research/upweight-the-strategy-not-the-tokens-faster-training-with-explicit-reasoning-thro/

BibTeX citation

```
@misc{oneill2025rgt,
  author       = {O'Neill, Charles and Partridge, Harry and Jayasekara,
Mudith},
  title        = {Upweight the Strategy, Not the Tokens: Faster Training with
Explicit Reasoning Through {RGT} (Rationale-Guided Training)},
  year         = {2025},
  month        = oct,
  day          = {29},
  howpublished = {Baseten Research},
  url          = {https://www.baseten.co/research/upweight-the-strategy-not-
the-tokens-faster-training-with-explicit-reasoning-thro/},
}
```

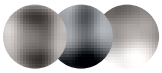
Other research



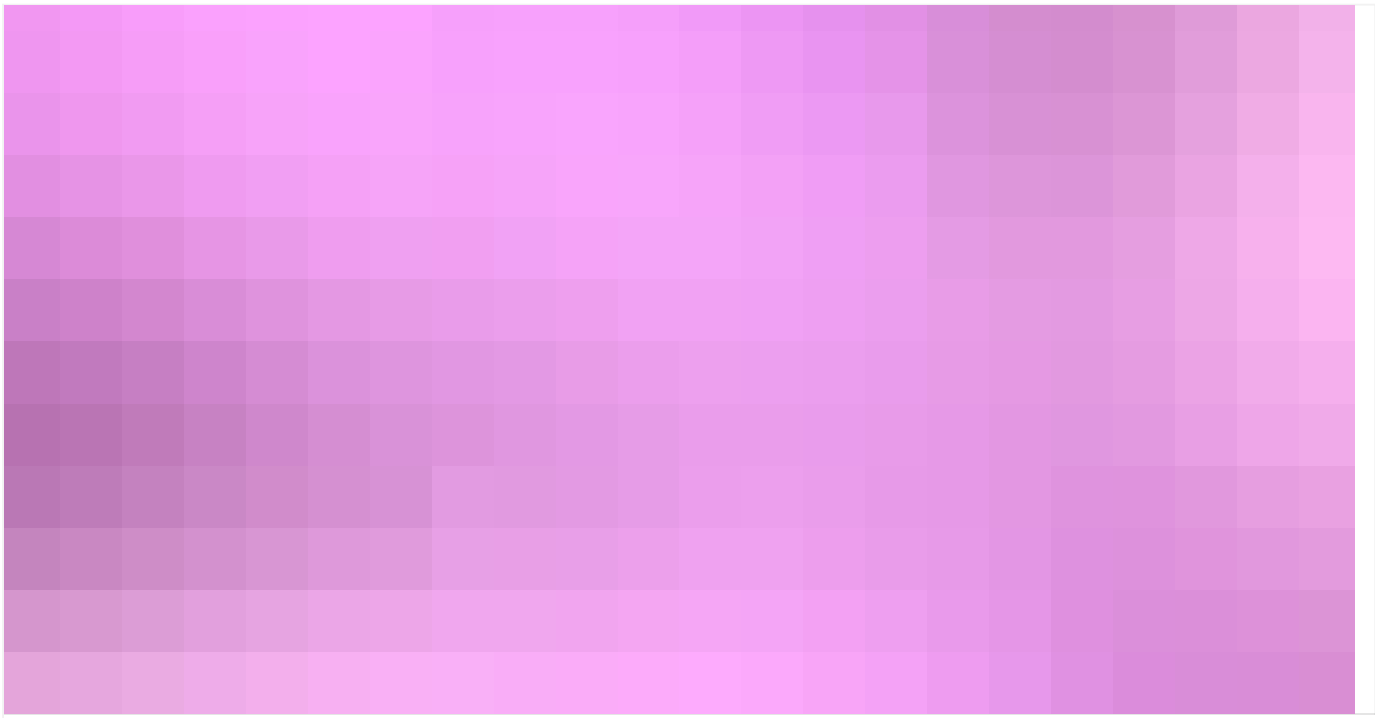
RESEARCH

Post-Training Science for Supervised Fine-Tuning

How the optimal learning rate and batch size move with model scale, family, and data, and whether one selection rule transfers across them.



CHARLES O'NEILL • 2 others



RESEARCH

Still: Amortized KV Cache Compaction in a Single Forward Pass

You can shrink a language model's KV cache by 200×, in a single forward pass, and it still answers correctly.

CHARLES O'NEILL • 4 others

Explore Baseten today

START DEPLOYING

TALK TO AN ENGINEER >

Product

Dedicated Inference
Model APIs
Training
Frontier Gateway

Baseten Inference Platform

Model Runtimes
Infrastructure
Multi-cloud

Deployment options

Cloud
Self-hosted
Hybrid

Embedded engineering

Forward deployed engineers

Modalities

Transcription
Image Generation
Text-to-speech
Large language models
Compound AI
Embeddings

Industries

Enterprise
Healthcare

Developer

Model library
Documentation
Changelog

Resources

Research
Customers
Blog
Guides
Events
Partners
Savings calculator
Trust Center
About us
Startups
Careers
Contact us

Popular models

GLM 5.2
Kimi K2.7 Code
DeepSeek V4
GPT OSS 120B
Whisper Large V3
NVIDIA Nemotron 3 Ultra
Explore all

Legal

Terms and Conditions
Privacy Policy
Service Level Agreement